CS110 / INDEPENDENCE ROAD

Working draft not an issued form / Lexxie

Issued: Nov 10, 2026, 1:20 PM Pacific

Record: WORKING-DRAFT

Personal public verification code: VIEW-9265F91D9B43EF8355AB19A9

Include your own public code with every assignment, even in a team folder. Never submit your private recovery key.

SHA-256: Not issued. No authentication number. Create the numbered forms in the assignment.

Verify the record number at courses.addp2p.com/development-road/independence/index.html?view=verify

An issued record preserves submitted information. It is not a grade or proof of comprehension.

prep

repo

https://github.com/Lexicon-Lexxie/CS110-Lexxie

gitTest

I made my folder inside my repository, with the structure with 12A, I added a plain txt file just so there is something in there for now.

access

the git is public, I plan on doing this project solo this time around.

video1

Observation:

I reviewed the pipeline workflow video and observed the importance of establishing a standardized project folder hierarchy and living documentation before writing any code or importing random assets.

Useful Idea & Interpretation:

A core idea is that a solid, predetermined folder structure combined with modular ReadMe/HTML files isn't just file management it is a team communication tool. When every team member knows exactly where models, scripts, audio, and documentation belong, onboarding is seamless and the risk of merge conflicts or misplaced dependencies is greatly reduced.

Question:

What naming conventions and folder templates does our team plan to agree upon so we avoid broken references when sharing assets in Unity/version control?

video2

Observation:

I reviewed the developer mindset video and observed that technical setbacks and bugs are framed not as personal shortcomings, but as normal, necessary milestones in the development cycle.

Useful Idea & Interpretation:

A core idea presented is treating failure as data rather than defeat. In software and game development, builds break and features fail routinely. Having a growth mindset means actively dissecting mistakes, embracing challenges outside our current comfort zone, and focusing on effort rather than fixed ability so that roadblocks turn into learning opportunities.

CS110 / INDEPENDENCE ROAD

Question:

When we run into code or design failures during sprints, what process will we use as a team to debrief and troubleshoot without placing blame?

video3

Observation:

I reviewed the folder and project setup demonstration and observed how establishing a dedicated directory structure (such as separating Context, Requirements, Resources, and dedicated game build folders) prevents clutter before any engine work begins.

Useful Idea & Interpretation:

A critical idea is structuring folders not just for personal convenience, but for team accessibility and onboarding. By utilizing JSON templates or standardized folder blueprints that include local ReadMe files and requirement checklists, any new team member can immediately understand where assets live, what each folder is for, and what prerequisites are needed without constant back-and-forth communication.

Question:

What core folder structure and root path naming rules will our team enforce before creating the shared Unity repository to ensure everyone stays consistent?

video4

Observation:

I reviewed the documentation video and observed that a Game Development Document (GDD) is treated not as a static, one-time chore, but as a living blueprint that evolves alongside the game as mechanics and milestones are tested.

Useful Idea & Interpretation:

A central idea is that documentation acts as the single source of truth for the entire team defining the narrative, core game loop, required art/audio assets, and individual member responsibilities. My interpretation is that having clear, centralized documentation prevents disputes and scope creep because every decision is measured against the agreed-upon design pillars rather than vague assumptions.

Question:

Who on our team will be responsible for keeping the GDD updated as we iterate, and where will it be hosted so everyone has real-time access?

video5

Observation:

I reviewed the introduction to Unity video and observed the core layout of the Unity Editor specifically the relationship between the Hierarchy (objects currently in the scene), Scene view (3D editor viewport), Game view (rendered player camera), Inspector (object properties and components), and the Project window (asset file structure).

Useful Idea & Interpretation:

CS110 / INDEPENDENCE ROAD

Mastering viewport navigation shortcuts (holding Alt + Left Click to orbit, Middle Click to pan, and pressing F to focus/frame a selected object) along with Ctrl + D to duplicate assets is essential for developer velocity. Being comfortable navigating 3D space and quickly switching between Scene and Game view prevents disorientation and allows team members to build and iterate on environments much faster.

Question:

What default editor layout and common tool settings should our team standardize so our screens and workflows look consistent during screen shares or pairing sessions?

video6

Observation:

I reviewed the custom player import video and observed how a 3D model (like an FBX downloaded from Mixamo) can immediately inherit existing controller logic and animations by setting its Rig Animation Type to Humanoid and assigning its new Avatar to the player's Animator component.

Useful Idea & Interpretation:

Unity s Humanoid rig system acts as a universal bone-mapping bridge. As long as our imported custom character and the template controller are both rigged as Humanoid avatars, we don't have to rebuild running, jumping, or sprint animations from scratch. This allows our art team to swap character models effortlessly while keeping core gameplay mechanics intact.

Question:

What character scale and bone naming conventions will our artists follow in Mixamo/Blender to ensure custom models retarget cleanly to Unity s Humanoid avatar system?

opinion

Observation:

Connecting our pipeline videos to my concept for StimuLye (where the protagonist "NaN" reconstructs a hospital and liminal memory rooms from total darkness), I decided that our team must strictly limit our initial build to the "Vertical Slice" demonstrated in the pipeline focusing only on the dark opening, one looping ward corridor, and one interactive memory room.

Useful Idea & Interpretation:

The pipeline's emphasis on modular Prefabs and scope management is the exact vehicle needed to make StimuLye playable. In the story, interacting with "Stimuli" objects restores stability to the dark void. By building our hospital walls, doors, and memory props as modular Unity prefabs with shared materials, we can dynamically spawn or trigger room transformations without having to hand-model endless unique rooms. Furthermore, formalizing this in our GDD keeps us from expanding into all four acts before the core sensory-restoration mechanic actually feels compelling.

Question:

How can we design our modular corridor and room prefabs so that a simple material or lighting swap can visually transition the hospital from normal to decaying liminal space as "Static" approaches?

questions

What I Did / Decided / Observed:

CS110 / INDEPENDENCE ROAD

I reviewed the complete preparation pipeline from developer mindset and folder architecture to GDD documentation, prefabs, and character rigging. Based on our past discussions around StimuLye, I decided that before committing to a final team role and project mission, we must resolve our core technical constraints, role boundaries, and MVP scope so we don't encounter friction or scope creep mid-sprint.

Questions to Resolve Before Committing:

Scope & MVP: Are we all in agreement to build strictly a tight vertical slice (the dark opening, one looping ward corridor, and one interactive memory room) rather than attempting all four acts right away?

Technical Pipeline: What specific Unity version (e.g., 2021.3 LTS), render pipeline (URP), and version control system (GitHub / Unity Version Control) are we locking in to avoid broken package dependencies?

Role Ownership: Who is taking primary ownership as Lead vs. Support across the essential pipeline roles (Programming/Player Controller, 3D Assets/Modular Prefabs, Sound Design, and GDD Documentation)?

currentPage

20

Name

Lexxie